



25/06/2026

**CESI**  
ÉCOLE D'INGÉNIEURS

# Projet Breezy

Livrable du bloc « Développement  
d'applications distribuées »

**CHALLIES Maéva**

CESI FISA A3 Informatique

Responsable pédagogique : M. Djamel AOUAM

## Table des matières

|                                                    |    |
|----------------------------------------------------|----|
| Introduction .....                                 | 2  |
| 1 Présentation et cadre du projet .....            | 3  |
| 1.1 Contexte et objectifs .....                    | 3  |
| 1.2 Analyse du besoin et périmètre retenu .....    | 3  |
| 1.3 Contraintes et priorisation .....              | 4  |
| 2 Organisation du projet .....                     | 5  |
| 2.1 Méthode de travail et planification .....      | 5  |
| 2.2 Outils et gestion du code source.....          | 6  |
| 3 Conception de l'application .....                | 7  |
| 3.1 Parcours utilisateur.....                      | 7  |
| 3.2 Architecture et répartition des services ..... | 8  |
| 3.3 Données, communications et sécurité.....       | 9  |
| 4 Réalisation technique.....                       | 10 |
| 4.1 Environnement, Docker et API Gateway .....     | 10 |
| 4.2 Service d'authentification.....                | 11 |
| 4.3 Service de profil .....                        | 11 |
| 4.4 Service de publication.....                    | 12 |
| 5 Validation de l'application.....                 | 13 |
| 5.1 Fonctionnalités et interfaces réalisées.....   | 13 |
| 5.2 Tests et scénario de démonstration .....       | 15 |
| 6 Bilan et perspectives .....                      | 17 |
| 6.1 Résultats, difficultés et limites.....         | 17 |
| 6.2 Améliorations envisagées.....                  | 18 |
| Conclusion .....                                   | 19 |
| Glossaire .....                                    | 20 |
| Table des figures.....                             | 21 |
| Table des tableaux.....                            | 22 |
| Annexes .....                                      | 23 |

# Introduction

Breezy est un projet réalisé dans le cadre du module « Développement d'applications distribuées ». L'objectif initial était de concevoir, en quatre semaines et en groupe de quatre personnes, un réseau social léger inspiré de Twitter/X. L'application devait notamment permettre aux utilisateurs de créer un compte, de gérer leur profil, de publier des messages courts et d'interagir avec les autres membres.

À la suite d'un changement d'organisation en fin de projet, la réalisation a été reprise individuellement. Afin de produire une version fonctionnelle et démontrable dans le temps disponible, le périmètre a été recentré sur un produit minimum viable réalisé en une journée. Cette nouvelle version conserve les principes essentiels du sujet, notamment l'architecture distribuée, la sécurisation des accès et la conteneurisation de l'application.

Le MVP repose ainsi sur trois services principaux : un service d'authentification, un service de profil et un service de publication. Il couvre un parcours utilisateur complet comprenant l'inscription, la connexion, la consultation et la modification du profil, la publication de messages courts, leur affichage sur le profil ainsi que la consultation d'un fil d'actualité global.

La problématique retenue est donc la suivante :

**Comment concevoir et réaliser, dans un délai fortement réduit, un parcours utilisateur fonctionnel reposant sur une architecture distribuée, sécurisée et conteneurisée ?**

Ce rapport présente d'abord le cadrage et l'organisation du projet. Il décrit ensuite la conception de l'application et sa réalisation technique. Enfin, il expose les fonctionnalités obtenues, les vérifications réalisées ainsi que les limites et les évolutions possibles du MVP.

# 1 Présentation et cadre du projet

## 1.1 Contexte et objectifs

Breezy est réalisé dans le cadre du module « Développement d'applications distribuées ». Le sujet initial prévoyait la conception, en quatre semaines et par un groupe de quatre personnes, d'un réseau social léger inspiré de Twitter/X.

L'application devait notamment permettre de créer un compte, gérer un profil, publier des messages courts et interagir avec les autres utilisateurs. Elle devait également mettre en œuvre une architecture distribuée, une authentification par JWT, une base de données, une interface responsive et une conteneurisation avec Docker.

À la suite d'un changement d'organisation en fin de projet, la réalisation a été reprise individuellement et recentrée sur une journée de développement. L'objectif final est de proposer un produit minimum viable cohérent, stable et démontrable, tout en conservant les principes essentiels d'une application distribuée.

## 1.2 Analyse du besoin et périmètre retenu

L'analyse du besoin a permis d'identifier les fonctionnalités indispensables à la création d'un parcours utilisateur cohérent.

Le MVP (Minimum Viable Product) couvre les fonctions nécessaires à un parcours utilisateur complet : création d'un compte, connexion, gestion du profil, publication de messages et consultation des publications.

Le périmètre final est présenté dans le tableau suivant.

| Fonctionnalité                       | Statut   | Justification                                         |
|--------------------------------------|----------|-------------------------------------------------------|
| Inscription et connexion             | Incluse  | Donne accès aux fonctions privées                     |
| Création et gestion du profil        | Incluse  | Permet d'identifier et de présenter l'utilisateur     |
| Publication limitée à 280 caractères | Incluse  | Constitue la fonction principale du réseau social     |
| Publications du profil               | Incluse  | Permet de retrouver les contenus publiés              |
| Fil d'actualité global               | Inclus   | Fournit une page d'accueil après la connexion         |
| Likes, commentaires et abonnements   | Différés | Nécessitent des règles et des données supplémentaires |
| Notifications, messagerie et médias  | Exclus   | Non indispensables au parcours principal              |
| Modération et administration         | Exclues  | Hors du périmètre prioritaire                         |

Tableau 1: Périmètre fonctionnel du MVP

### 1.3 Contraintes et priorisation

La principale contrainte est le temps de réalisation, limité à une journée pour la mise en œuvre technique. La réalisation individuelle implique également de prendre en charge le cadrage, le développement, l'intégration et la validation de l'ensemble de l'application.

Le caractère distribué du projet devait néanmoins être conservé. Le MVP repose donc sur trois services distincts, complétés par un frontend, une API Gateway et une base de données conteneurisée.

Les fonctionnalités ont été réparties de manière informelle en trois niveaux de priorité.

| Priorité             | Éléments concernés                                            | Objectif                                         |
|----------------------|---------------------------------------------------------------|--------------------------------------------------|
| <b>Indispensable</b> | Docker, API Gateway, authentification, profil et publications | Obtenir une application distribuée fonctionnelle |
| <b>Importante</b>    | Frontend, fil global, validations et gestion des erreurs      | Rendre le parcours utilisable et démontrable     |
| <b>Différée</b>      | Interactions sociales, médias et rôles avancés                | Limiter la complexité du MVP                     |

Tableau 2: Priorisation du projet

## 2 Organisation du projet

### 2.1 Méthode de travail et planification

Le développement est organisé en blocs fonctionnels courts correspondant aux principales parties de l'application :

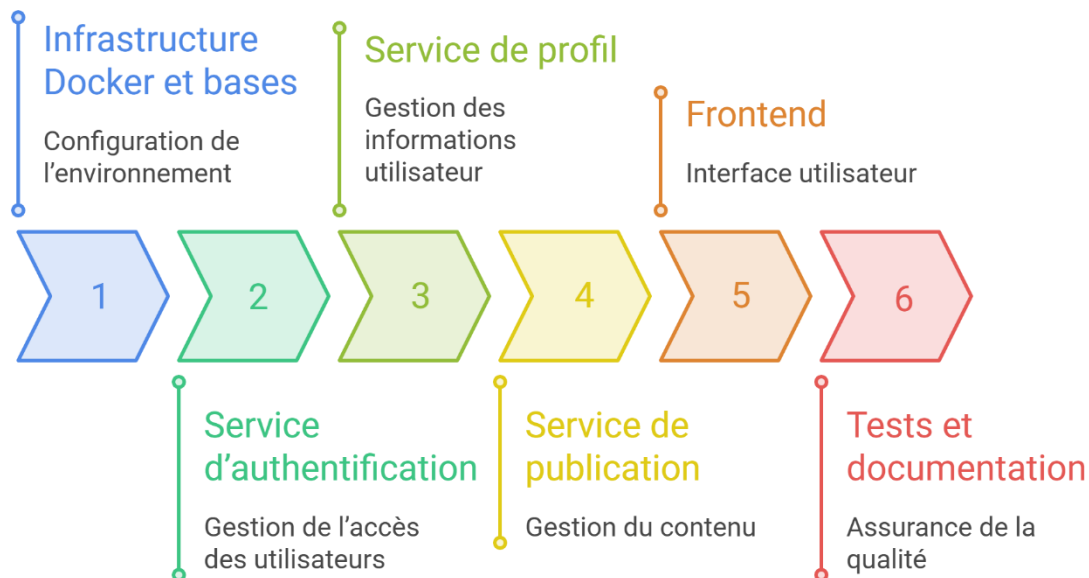


Figure 1: Plan d'action de la réalisation technique

Un fichier `TODO.md` sert de support de suivi. Il est mis à jour à la fin de chaque étape afin de distinguer les tâches terminées de celles restant à réaliser.

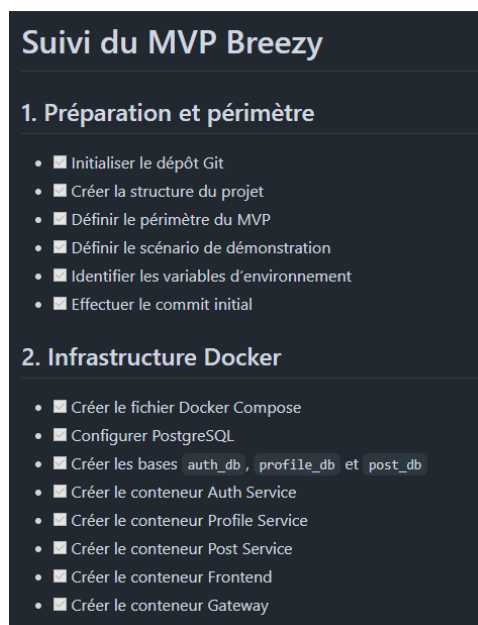


Figure 2: Extrait visuel du fichier `TODO.md`

La validation associe des tests fonctionnels manuels, des tests unitaires automatisés, la vérification des données enregistrées, la consultation du frontend et l'analyse ponctuelle des logs Docker.

## 2.2 Outils et gestion du code source

Plusieurs outils ont été utilisés pour organiser le développement, exécuter l'application et suivre les modifications.

| Outil                            | Utilisation                                             |
|----------------------------------|---------------------------------------------------------|
| Visual Studio Code               | Développement et consultation du code                   |
| Docker Desktop et Docker Compose | Construction et exécution des conteneurs                |
| PowerShell                       | Commandes de lancement et tests des API                 |
| Git et GitHub                    | Versionnement, hébergement et pull requests             |
| GitHub Desktop                   | Consultation visuelle des modifications                 |
| PostgreSQL                       | Stockage des comptes, profils et publications           |
| Navigateur web                   | Vérification de l'interface et du responsive            |
| Node.js et npm                   | Exécution des services et lancement des tests unitaires |

Tableau 3: Principaux outils utilisés

Le code est organisé autour d'une branche par fonctionnalité principale. Après le développement et les vérifications, une pull request est créée avant la fusion dans la branche main.

Les commits suivent une convention simple, par exemple :

- feat(auth): add registration login and JWT authentication
- feat(profile): add profile consultation and update
- feat(posts): add post creation and listing

Cette organisation conserve un historique lisible et permet d'isoler les modifications apportées à chaque composant.

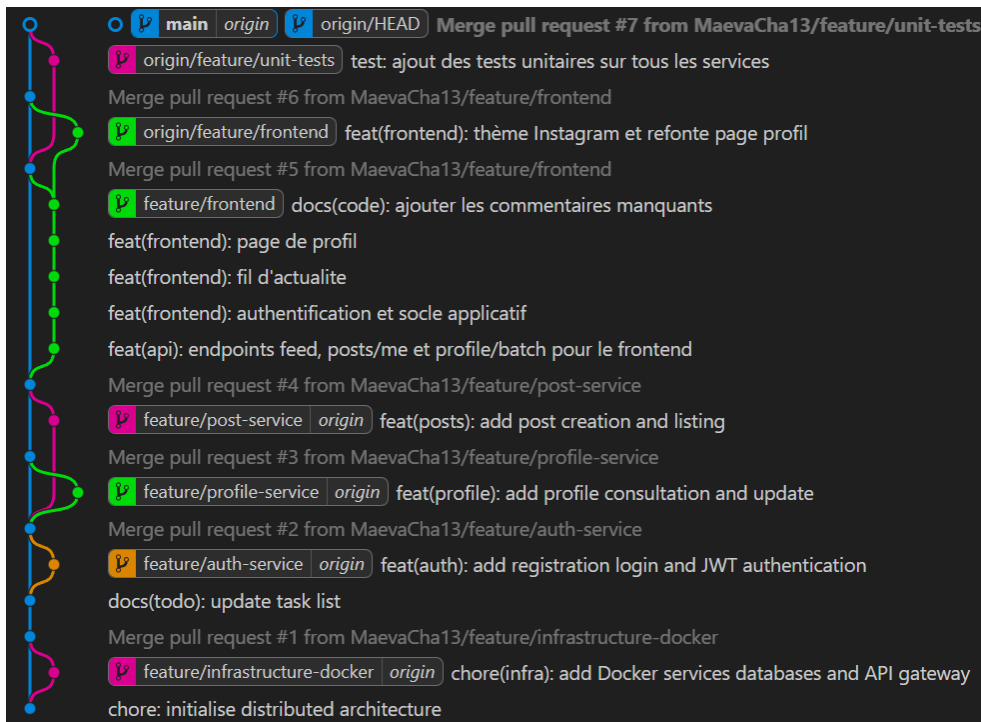


Figure 3: Graphe de l'architecture git

## 3 Conception de l'application

### 3.1 Parcours utilisateur

Lors de l'inscription, le visiteur renseigne son nom d'affichage, son adresse électronique et son mot de passe. Le compte est créé dans le service d'authentification, puis un profil associé est automatiquement généré.

Lors de la connexion, les identifiants sont vérifiés. Lorsque ceux-ci sont valides, un JWT est retourné au frontend et permet d'accéder aux pages protégées.

Depuis sa page de profil, l'utilisateur peut consulter et modifier son nom, sa biographie et l'URL de son avatar. Ses publications sont également affichées sur cette page.

L'utilisateur authentifié peut publier un message limité à 280 caractères. La publication est enregistrée avec son identifiant et apparaît dans le fil global ainsi que sur son profil.

La déconnexion supprime le jeton conservé par le frontend. Toute nouvelle tentative d'accès à une page protégée entraîne alors une redirection vers la page de connexion.

Le scénario utilisateur de référence est le suivant :

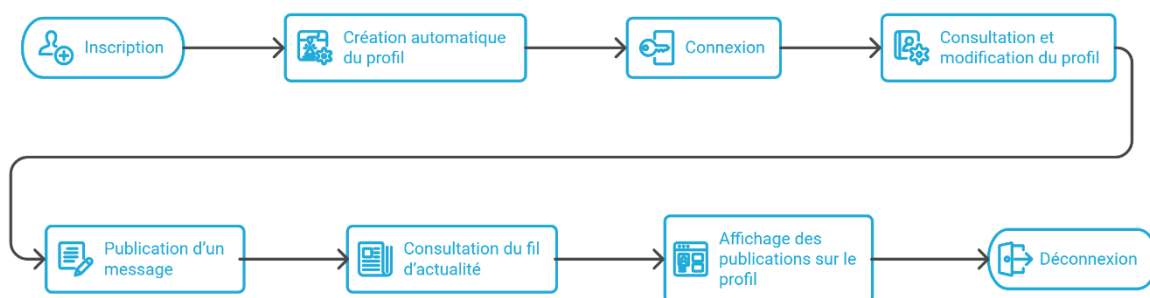


Figure 4 : Parcours utilisateur de référence



## 3.2 Architecture et répartition des services

Breezy repose sur une architecture distribuée comprenant un frontend, une API Gateway et trois services backend indépendants.

| Composant                | Responsabilité                                         |
|--------------------------|--------------------------------------------------------|
| <b>Frontend Next.js</b>  | Affichage des pages et interactions avec l'utilisateur |
| <b>API Gateway Nginx</b> | Point d'entrée unique et routage des requêtes          |
| <b>Auth Service</b>      | Gestion des comptes, des connexions et des JWT         |
| <b>Profile Service</b>   | Création, consultation et modification des profils     |
| <b>Post Service</b>      | Création et récupération des publications              |
| <b>PostgreSQL</b>        | Hébergement des trois bases de données                 |

Tableau 4: Répartition des composants

Le frontend communique uniquement avec l'API Gateway. Celle-ci redirige les requêtes selon leur chemin :

- /api/auth → Auth Service
- /api/profile → Profile Service
- /api/posts → Post Service

Chaque service dispose de sa propre base logique :

- Auth Service → auth\_db
- Profile Service → profile\_db
- Post Service → post\_db

Lors de l'inscription, l'Auth Service appelle le Profile Service afin de créer automatiquement le profil du nouvel utilisateur.

Un seul conteneur PostgreSQL héberge les trois bases afin de simplifier l'environnement de développement. Les données restent toutefois séparées et chaque service accède uniquement à sa propre base.

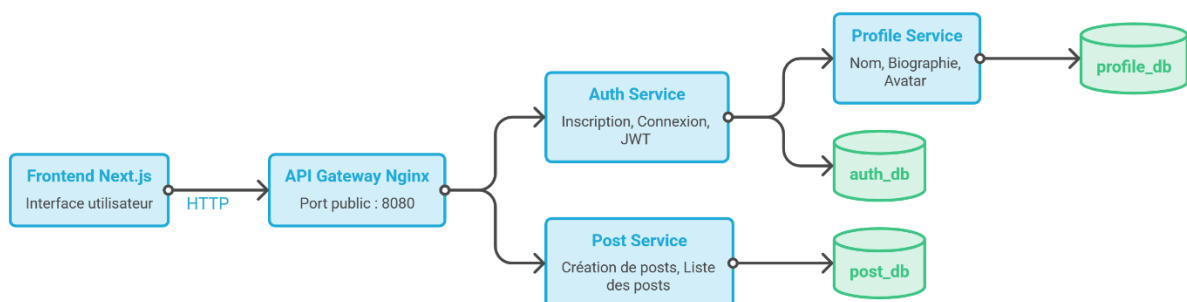


Figure 5: Schéma de l'architecture distribuée

### 3.3 Données, communications et sécurité

Les données sont réparties entre trois tables principales.

| Base       | Table    | Données principales                                                 |
|------------|----------|---------------------------------------------------------------------|
| auth_db    | users    | Identifiant, courriel, mot de passe haché, rôle et date de création |
| profile_db | profiles | Identifiant utilisateur, nom, biographie, avatar et dates           |
| post_db    | posts    | Identifiant, auteur, contenu et dates de publication                |

Tableau 5: Organisation des données

Les identifiants sont générés au format UUID. Aucune clé étrangère n'est créée entre les bases : les services conservent ainsi la propriété de leurs données.

Deux types de communication sont utilisés :

**Flux public :** Frontend → API Gateway → service concerné

**Flux interne :** Auth Service → Profile Service lors de l'inscription

L'appel interne est protégé par une clé transmise dans l'en-tête X-Internal-API-Key.

Les principales mesures de sécurité mises en œuvre sont :

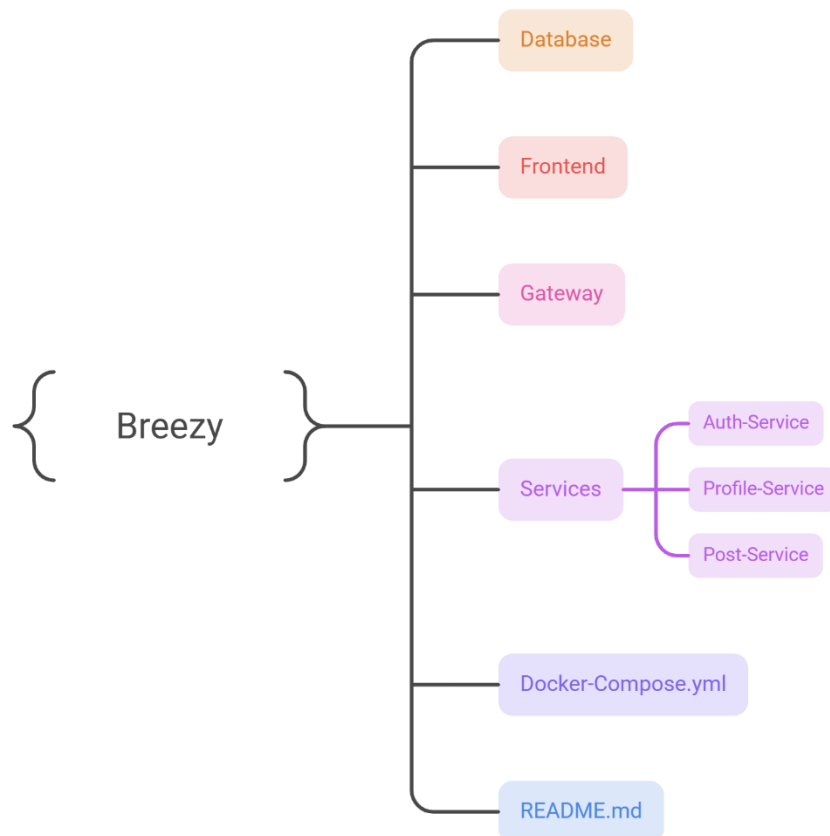
- le hachage des mots de passe avec bcrypt
- la signature et l'expiration des JWT
- la récupération de l'utilisateur depuis le champ sub du jeton
- la protection des routes privées par un middleware
- la validation des données reçues
- le stockage des secrets dans les variables d'environnement
- l'utilisation de messages génériques en cas d'échec de connexion

## 4 Réalisation technique

### 4.1 Environnement, Docker et API Gateway

Les services backend sont développés avec Node.js et Express. Le frontend utilise Next.js et React. PostgreSQL stocke les données, tandis que Nginx joue le rôle d'API Gateway.

L'organisation principale du dépôt est la suivante :



*Figure 6: Structure du dépôt de l'application*

Docker Compose construit et démarre PostgreSQL, les trois services backend, Nginx et le frontend. Les composants partagent un réseau Docker et les données PostgreSQL sont conservées dans un volume.

L'application est lancée avec la commande suivante :

« `docker compose up --build` »

Nginx expose le port 8080 et redirige les requêtes vers le service concerné. Le frontend est accessible sur le port 3000. Les ports des services et de PostgreSQL restent internes au réseau Docker.

Le frontend contient les pages d'accueil, d'inscription, de connexion, de fil d'actualité et de profil. Un client HTTP centralise les appels aux API, ajoute le JWT aux requêtes protégées et gère les réponses d'erreur.

## 4.2 Service d'authentification

L'Auth Service prend en charge la création des comptes et l'accès sécurisé à l'application. Il utilise Express et communique uniquement avec la base auth\_db.

| Méthode | Route              | Rôle                             |
|---------|--------------------|----------------------------------|
| GET     | /health            | Vérifier l'état du service       |
| POST    | /api/auth/register | Créer un compte                  |
| POST    | /api/auth/login    | Connecter un utilisateur         |
| GET     | /api/auth/me       | Récupérer l'utilisateur connecté |
| POST    | /api/auth/logout   | Confirmer la déconnexion         |

Tableau 6: Routes principales de l'Auth Service

Lors de l'inscription, le service vérifie le courriel, le nom d'affichage et la longueur du mot de passe. L'adresse est normalisée, un UUID est généré et le mot de passe est haché avant son enregistrement.

Le service demande ensuite au Profile Service de créer le profil associé. En cas d'échec de cette opération, le compte nouvellement créé est supprimé afin d'éviter une donnée incomplète.

Lors de la connexion, le mot de passe saisi est comparé au hachage enregistré. Un JWT contenant l'identifiant, le courriel, le rôle et une date d'expiration est retourné lorsque les identifiants sont valides.

Le code est séparé entre routes, contrôleurs, logique métier, accès aux données, middleware JWT et client interne.

## 4.3 Service de profil

Le Profile Service gère les informations personnelles associées à chaque utilisateur. Il communique uniquement avec la base profile\_db.

| Méthode | Route              | Rôle                                |
|---------|--------------------|-------------------------------------|
| GET     | /health            | Vérifier l'état du service          |
| POST    | /internal/profiles | Créer automatiquement un profil     |
| GET     | /api/profile/me    | Consulter son profil                |
| PATCH   | /api/profile/me    | Modifier son profil                 |
| GET     | /api/profile/batch | Récupérer plusieurs profils publics |

Tableau 7: Routes principales du Profile Service

La route interne de création est appelée par l'Auth Service et protégée par une clé API. Les autres routes utilisent le JWT : l'identifiant du profil provient du champ sub et ne peut pas être choisi librement par le frontend.

L'utilisateur peut modifier son nom, sa biographie et l'URL de son avatar. Le nom est limité à 50 caractères, la biographie à 160 caractères et l'avatar à une URL HTTP ou HTTPS.

La route de récupération groupée permet d'obtenir les noms et avatars des auteurs affichés dans le fil global.

## 4.4 Service de publication

Le Post Service gère la création et l’affichage des publications. Il utilise la base `post_db` et n’accède pas directement aux données des comptes ou des profils.

| Méthode | Route         | Rôle                       |
|---------|---------------|----------------------------|
| GET     | /health       | Vérifier l’état du service |
| POST    | /api/posts/   | Créer une publication      |
| GET     | /api/posts/me | Récupérer ses publications |
| GET     | /api/posts/   | Récupérer le fil global    |

*Tableau 8: Routes principales du Post Service*

Les routes fonctionnelles sont protégées par JWT. Lors de la création d’un message, l’identifiant de l’auteur est récupéré depuis le jeton.

Le contenu est nettoyé avant son enregistrement. Un message absent, vide ou supérieur à 280 caractères est refusé. Les publications sont ensuite triées de la plus récente à la plus ancienne.

Le Post Service retourne les identifiants des auteurs. Le frontend récupère leurs informations publiques auprès du Profile Service puis assemble les données nécessaires à l’affichage du fil.

## 5 Validation de l'application

### 5.1 Fonctionnalités et interfaces réalisées

Le MVP permet d'exécuter l'ensemble du parcours retenu depuis une interface responsive.

Les écrans principaux sont :

- la page d'accueil publique
- le formulaire d'inscription
- le formulaire de connexion
- le fil d'actualité global
- le formulaire de publication
- la page de profil
- le formulaire de modification du profil

Voici ci-dessous le rendu visuel de quelques écrans :

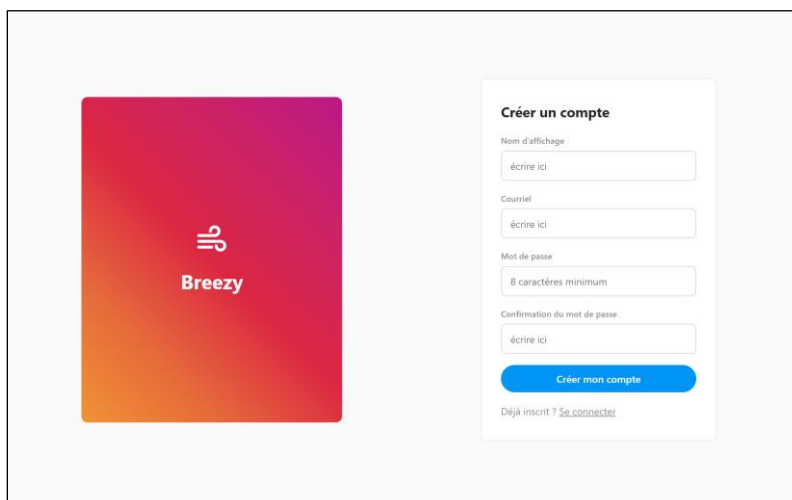


Figure 7: Page d'inscription

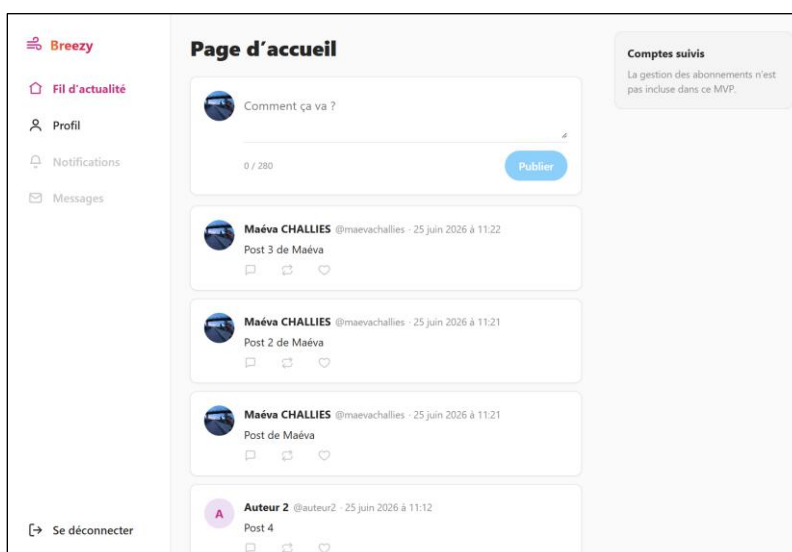


Figure 8: Page d'accueil et fil d'actualité

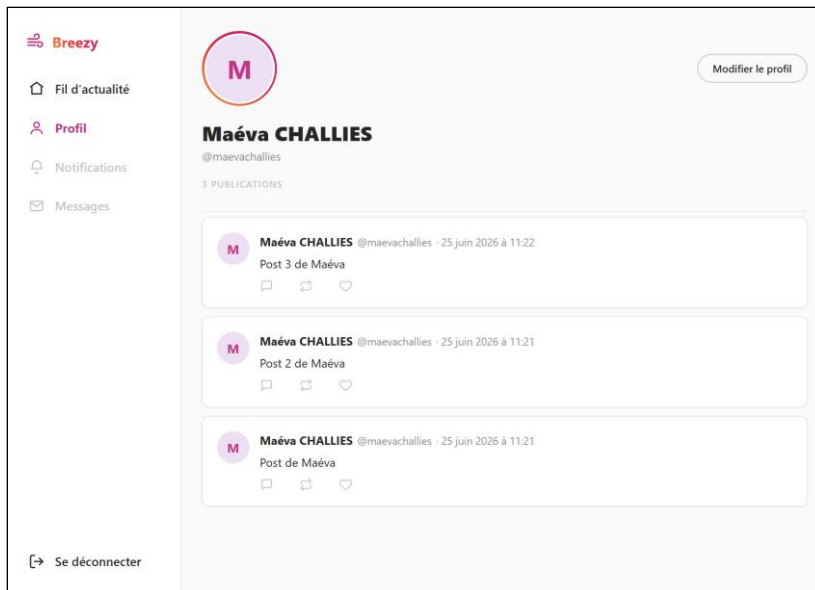


Figure 9: Page de profil



Figure 10: Page d'accueil et fil d'actualité en version mobile

| Fonctionnalité                | État final     | Limite principale                                |
|-------------------------------|----------------|--------------------------------------------------|
| Inscription                   | Fonctionnelle  | Pas de validation par courriel                   |
| Connexion par JWT             | Fonctionnelle  | Pas de renouvellement automatique                |
| Création et gestion du profil | Fonctionnelle  | Avatar fourni par URL                            |
| Création d'une publication    | Fonctionnelle  | Contenu textuel uniquement                       |
| Publications du profil        | Fonctionnelles | Pas de modification ni de suppression            |
| Fil global                    | Fonctionnel    | Non personnalisé selon les abonnements           |
| Déconnexion                   | Fonctionnelle  | Jeton supprimé uniquement côté client            |
| Responsive                    | Fonctionnel    | Vérification réalisée dans le navigateur         |
| Interactions sociales         | Non réalisées  | Actions éventuellement visibles mais désactivées |

Tableau 9: État final des fonctionnalités

Les mockups fournis dans le sujet ont servi de base pour les écrans d'authentification, d'accueil et de profil. Le résultat final reprend leur organisation générale tout en limitant les actions disponibles au périmètre du MVP.

## 5.2 Tests et scénario de démonstration

La validation repose sur des vérifications fonctionnelles manuelles et sur des tests unitaires automatisés exécutés dans le frontend et les trois services backend.

L'application est d'abord recréée depuis un environnement vide :

- docker compose down -v
- docker compose up --build

La commande docker « compose ps » et les routes de contrôle permettent ensuite de vérifier la disponibilité des composants.

| Test                                  | Résultat attendu              | Statut |
|---------------------------------------|-------------------------------|--------|
| Inscription valide                    | Compte et profil créés        | Validé |
| Courriel déjà utilisé                 | Inscription refusée           | Validé |
| Connexion valide                      | JWT retourné                  | Validé |
| Mauvais mot de passe                  | Connexion refusée             | Validé |
| Accès sans JWT                        | Accès refusé ou redirection   | Validé |
| Modification valide du profil         | Données mises à jour          | Validé |
| Biographie trop longue                | Modification refusée          | Validé |
| Publication vide                      | Publication refusée           | Validé |
| Publication de plus de 280 caractères | Publication refusée           | Validé |
| Deux publications successives         | Tri chronologique respecté    | Validé |
| Déconnexion                           | Jeton supprimé et redirection | Validé |
| Recréation de l'environnement         | Application opérationnelle    | Validé |

Tableau 10: Synthèse des vérifications fonctionnelles manuelles

Les tests unitaires utilisent le module de test intégré à Node.js et sont exécutés avec la commande npm test. Au total, 80 tests ont été exécutés avec succès, sans aucun échec. Ils vérifient principalement les validations de données, les middlewares de sécurité et certains utilitaires du frontend.



| Composant              | Principaux éléments vérifiés                                                       | Résultat     |
|------------------------|------------------------------------------------------------------------------------|--------------|
| <b>Auth Service</b>    | Courriel, mot de passe, nom d'affichage, validation des JWT et gestion des erreurs | 21/21        |
| <b>Profile Service</b> | JWT, clé API interne, filtrage des identifiants et champs modifiables              | 32/32        |
| <b>Post Service</b>    | JWT, contenu obligatoire, nettoyage et limite de 280 caractères                    | 11/11        |
| <b>Frontend</b>        | Formatage des dates et génération des identifiants affichés                        | 16/16        |
| <b>Total</b>           |                                                                                    | <b>80/80</b> |

*Tableau 11: Synthèse des tests unitaires*

Ces tests restent des tests unitaires ciblés. Les communications réelles entre les services, les accès à PostgreSQL et le parcours complet depuis le frontend sont principalement vérifiés par les tests fonctionnels manuels.

La démonstration finale suit un scénario unique :

1. démarrage de l'application
2. création d'un compte
3. connexion
4. consultation du profil créé automatiquement
5. modification de la biographie et de l'avatar
6. publication de deux messages
7. vérification du fil global et du profil
8. déconnexion et contrôle de la protection des pages

Ce scénario permet de présenter la chaîne complète : frontend, API Gateway, services backend, JWT et bases de données.

## 6 Bilan et perspectives

### 6.1 Résultats, difficultés et limites

Le MVP obtenu repose sur trois services backend, trois bases logiquement séparées, une API Gateway, une authentification par JWT et une interface responsive. Le parcours principal est utilisable sans intervention manuelle dans les bases. Une suite de 80 tests unitaires automatisés complète les vérifications fonctionnelles du parcours principal.

Les principales difficultés rencontrées concernent la reprise individuelle dans un délai réduit, la coordination des différents composants et l'intégration du fil global. Le développement progressif des services et leur validation étape par étape ont permis de limiter les problèmes d'intégration.

Les choix techniques ont été adaptés au contexte : un seul conteneur PostgreSQL, des communications HTTP synchrones, un avatar fourni par URL et une déconnexion gérée côté frontend. Ces simplifications conservent les principes de l'architecture distribuée tout en réduisant la charge de développement.

La version actuelle présente néanmoins plusieurs limites :

- absence de likes, commentaires et abonnements
- fil global non personnalisé
- absence de modification et de suppression des publications
- profils publics limités
- un seul rôle réellement exploité
- publications uniquement textuelles
- absence de révocation et de renouvellement du JWT
- absence de tests automatisés d'intégration et de bout en bout
- infrastructure principalement adaptée au développement

## 6.2 Améliorations envisagées

Les améliorations sont classées selon leur importance pour le parcours utilisateur et leur cohérence avec les fonctionnalités initialement attendues.

| Priorité | Amélioration                                               | Intérêt                                                                                 |
|----------|------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Haute    | Ajouter des tests d'intégration et de bout en bout         | Vérifier automatiquement les échanges entre services et le parcours utilisateur complet |
| Haute    | Ajouter la modification et la suppression des publications | Donner plus de contrôle à l'auteur                                                      |
| Haute    | Renforcer la gestion des JWT                               | Améliorer la sécurité des sessions                                                      |
| Moyenne  | Ajouter les likes et les commentaires                      | Développer les interactions sociales                                                    |
| Moyenne  | Ajouter les abonnements et un fil personnalisé             | Se rapprocher du fonctionnement d'un réseau social                                      |
| Moyenne  | Ajouter les profils publics                                | Faciliter la navigation entre utilisateurs                                              |
| Basse    | Ajouter le téléversement d'images                          | Enrichir les profils et les publications                                                |
| Basse    | Ajouter la modération et l'administration                  | Gérer les utilisateurs et les contenus                                                  |
| Basse    | Mettre en place une intégration continue                   | Automatiser les tests à chaque modification                                             |

Tableau 12: Principales améliorations envisagées

Les évolutions prioritaires concernent d'abord la fiabilité et la sécurité du socle existant. Les fonctions sociales pourront ensuite être ajoutées progressivement sans remettre en cause la séparation actuelle des services.

## Conclusion

Le projet Breezy avait pour objectif de mettre en œuvre un réseau social reposant sur une architecture distribuée. Dans le temps disponible, le périmètre a été recentré sur un MVP composé de trois services dédiés à l'authentification, au profil et aux publications.

Le parcours principal est fonctionnel : un utilisateur peut créer un compte, se connecter, modifier son profil, publier des messages, consulter le fil global et se déconnecter. Docker Compose et l'API Gateway permettent de lancer et d'utiliser l'ensemble de manière centralisée. De plus, ce parcours principal est complété par 80 tests unitaires automatisés couvrant les principaux contrôles de validation et de sécurité.

Le résultat reste volontairement limité et ne comprend pas encore les principales interactions sociales. Il permet cependant de démontrer la séparation des responsabilités, la communication entre services, la sécurisation des accès et la conteneurisation d'une application distribuée.

## Glossaire

| Terme                                          | Définition                                                                                                            |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>API (Application Programming Interface)</b> | Interface permettant à plusieurs applications ou services d'échanger des données au moyen de requêtes.                |
| <b>API Gateway</b>                             | Point d'entrée unique qui reçoit les requêtes du frontend et les redirige vers le service concerné.                   |
| <b>Backend</b>                                 | Partie serveur de l'application, chargée du traitement des requêtes, de la logique métier et de l'accès aux données.  |
| <b>Conteneur</b>                               | Environnement isolé regroupant une application et ses dépendances afin de faciliter son exécution et son déploiement. |
| <b>Frontend</b>                                | Partie visible de l'application avec laquelle l'utilisateur interagit depuis son navigateur.                          |
| <b>Hachage</b>                                 | Transformation irréversible d'une donnée. Dans Breezy, les mots de passe sont hachés avant leur stockage.             |
| <b>JWT (JSON Web Token)</b>                    | Jeton signé contenant des informations sur l'utilisateur et permettant d'accéder aux routes protégées.                |
| <b>Middleware</b>                              | Fonction exécutée pendant le traitement d'une requête, par exemple pour vérifier la validité d'un JWT.                |
| <b>MVP (Minimum Viable Product)</b>            | Version réduite d'une application contenant uniquement les fonctionnalités essentielles.                              |
| <b>Service</b>                                 | Composant autonome chargé d'un domaine précis. Breezy sépare l'authentification, les profils et les publications.     |
| <b>UUID</b>                                    | Identifiant unique utilisé pour distinguer les utilisateurs et les publications.                                      |

Tableau 13: Glossaire du livrable

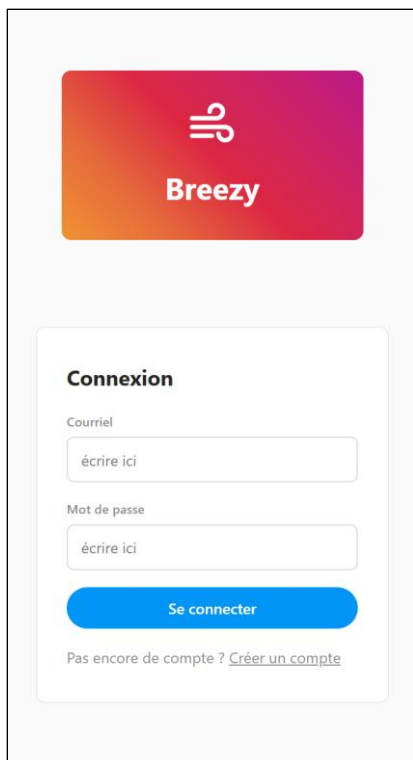
## Table des figures

|                                                                     |    |
|---------------------------------------------------------------------|----|
| Figure 1: Plan d'action de la réalisation technique.....            | 5  |
| Figure 2: Extrait visuel du fichier TODO.md .....                   | 5  |
| Figure 3: Graphe de l'architecture git .....                        | 6  |
| Figure 4 : Parcours utilisateur de référence .....                  | 7  |
| Figure 5: Schéma de l'architecture distribuée .....                 | 8  |
| Figure 6: Structure du dépôt de l'application.....                  | 10 |
| Figure 7: Page d'inscription .....                                  | 13 |
| Figure 8: Page d'accueil et fil d'actualité .....                   | 13 |
| Figure 9: Page de profil.....                                       | 14 |
| Figure 10: Page d'accueil et fil d'actualité en version mobile..... | 14 |
| Figure 11: Page de connexion en version mobile .....                | 23 |
| Figure 12: Page d'inscription en version mobile.....                | 23 |
| Figure 13: Page de profil après modification.....                   | 24 |

## Table des tableaux

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Tableau 1: Périmètre fonctionnel du MVP .....                         | 3  |
| Tableau 2: Priorisation du projet .....                               | 4  |
| Tableau 3: Principaux outils utilisés .....                           | 6  |
| Tableau 4: Répartition des composants .....                           | 8  |
| Tableau 5: Organisation des données .....                             | 9  |
| Tableau 6: Routes principales de l'Auth Service .....                 | 11 |
| Tableau 7: Routes principales du Profile Service .....                | 11 |
| Tableau 8: Routes principales du Post Service .....                   | 12 |
| Tableau 9: État final des fonctionnalités .....                       | 15 |
| Tableau 10: Synthèse des vérifications fonctionnelles manuelles ..... | 15 |
| Tableau 11: Synthèse des tests unitaires .....                        | 16 |
| Tableau 12: Principales améliorations envisagées.....                 | 18 |
| Tableau 13: Glossaire du livrable .....                               | 20 |

## Annexes



The image shows a mobile login page for the Breezy application. At the top, there is a Breezy logo consisting of a stylized 'B' icon and the word 'Breezy' on a red-to-orange gradient background. Below the logo, the page is titled 'Connexion'. It features two input fields: 'Courriel' (Email) and 'Mot de passe' (Password), both with placeholder text 'écrire ici'. A blue button labeled 'Se connecter' is positioned below the password field. At the bottom, there is a link that reads 'Pas encore de compte ? [Créer un compte](#)'.

Figure 11: Page de connexion en version mobile



The image shows a mobile registration page for the Breezy application. At the top, there is a Breezy logo consisting of a stylized 'B' icon and the word 'Breezy' on a red-to-orange gradient background. Below the logo, the page is titled 'Créer un compte'. It features four input fields: 'Nom d'affichage' (Display Name), 'Courriel' (Email), 'Mot de passe' (Password), and 'Confirmation du mot de passe' (Confirm Password). The password field has a placeholder '8 caractères minimum'. A blue button labeled 'Créer mon compte' is positioned below the confirmation field. At the bottom, there is a link that reads 'Déjà inscrit ? [Se connecter](#)'.

Figure 12: Page d'inscription en version mobile



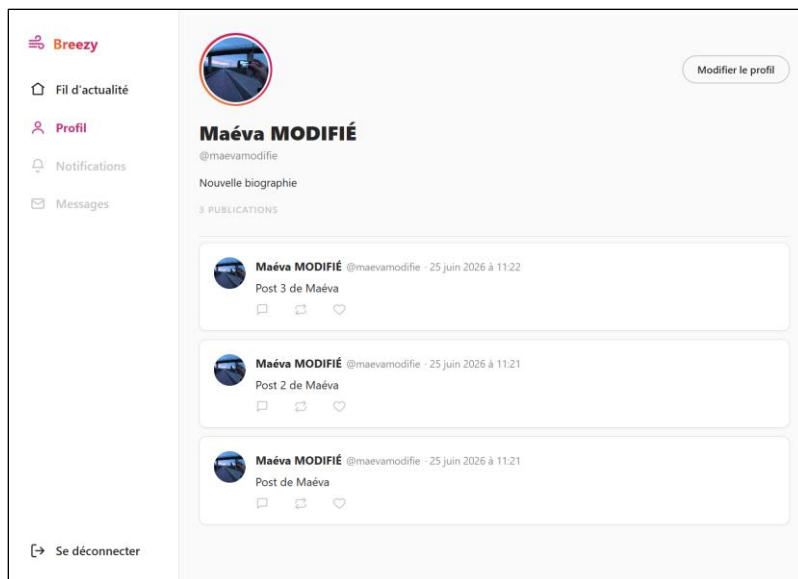


Figure 13: Page de profil après modification